CrossTwin: Function and Equation Registry

A traceability record between the DPSIR causal graph and its computational implementation

CrossTwin Project

October 13, 2026

Abstract

This document catalogs every quantitative computation currently implemented in the CrossTwin platform's domain apps (`administrative`, `physicalEnv`, `watersupply`, `housing`, `urban_heat`), plus the map-editing and what-if backup functions of `mainMap`, records the exact equation each function evaluates, the variables and their units, the source data model(s), and the corresponding edge(s) in the project's Driver–Pressure–State–Impact–Response (DPSIR) causal graph (`core/DAG.dot`). It is intended as a working reference while preparing a manuscript describing CrossTwin's indicator methodology, and as a running record of which causal relationships are backed by executable derivation logic versus flat, manually entered fields. Status tags (implemented / partial / gap) follow the coverage audit in `TODO.md`.

Contents

1 Scope and conventions **3**

2 Administrative app — population projection **3**

2.1 Compound annual growth rate — `extrapolate()` . . . 3
2.2 City-level growth factor — `_city_growth_factor()` . . . 4
2.3 District/Neighbourhood projection — `population_by_year()` . . . 4
2.4 Province projection . . . 4
2.5 What-if growth adjustment — `_adjust()` . . . 4
2.6 City annual growth rate — `annual_growth_rate()` . . . 5
2.7 Population and density cascade — `City.save()`, `Province.save()` . . . 5
2.8 Urban area cascade and source computation — `administrative/signals.py`, `physicalEnv/signals.py` 6
2.9 Urban-fabric buffer around new buildings — `urbanize_around()`, `building_created` 6

3 Physical environment app — soil hydrology and infiltration **6**

3.1 Hydrologic soil group — `classify_soil_group()`, `SoilType.save()` . . . 6
3.2 Curve number — `curve_number()` . . . 7
3.3 Shallow groundwater and drained land — `soil_landcover_composition()` . . . 8
3.4 SCS runoff and infiltration coefficient — `scs_runoff_mm()`, `infiltration_coefficient()` 9
3.5 Area-weighted unit summary — `summarize_infiltration()` . . . 9
3.6 HILUCS land-use codes — `parse_hilucs_href()`, `hilucs_q()` . . . 9

4 Water supply app **10**
4.1 Per-capita consumption — `_get_consumption_capita()` . . . 10
4.2 Total water demand — `calculate_total_demand()`, `TotalWaterDemand.save()` . . 10
4.3 Total consumption — `ConsumptionCapita.save()` . . . 10
4.4 Total extraction — `calculate_total_extraction()` . . . 10
4.5 Total production — `calculate_total_production_day()` . . . 11
4.6 Supply security & service time . . . 11
4.7 Energy consumption — `calculate_energy_consumption()` . . . 11
4.8 Pump emissions — `ExtractionWater.save()` . . . 11
4.9 Extraction OPEX and environmental cost — `ExtractionWater.save()` . . . 11
4.10 Consolidated OPEX — `OPEX.save()` . . . 12
4.11 OPEX recovery (dashboard) — `calculate_opex_recovery()` . . . 12
4.12 Billed consumption and revenue — `MeteredResidential.save()` . . . 12
4.13 Collection ratio — `calculate_collection_ratio()` . . . 12
4.14 Coverage — `calculate_coverage()`, `CoverageWaterSupply.save()` . . . 13
4.15 Non-Revenue Water and ILI . . . 13
4.16 Water quality — `calculate_water_quality()` . . . 13
4.17 Available fresh water and drought area . . . 14
4.18 Infiltration indicator (dashboard) — `calculate_infiltration()`, `_infiltration_indicators()` 14
4.19 Pipe geometry — `PipeNetwork.save()` . . . 14

5 Housing app **14**
5.1 Supply/demand deficit . . . 15
5.2 Pipeline units — `calculate_new_units()` . . . 15
5.3 Mortgage indicators — `calculate_mortgage_indicators()` . . . 15
5.4 House Price Index — `calculate_house_price_index()` . . . 15
5.5 Affordability index — `HousingAffordability.save()` . . . 16
5.6 Zoning composition — `calculate_zoning()` . . . 16
5.7 Property market and vacancy — `calculate_property_indicators()` . . . 16

6 Urban heat app **16**
6.1 Green area and vegetation coverage . . . 17
6.2 Urban morphology / canyon aspect proxy — `calculate_urban_morphology()` . . . 17
6.3 Raster statistics — `_raster_stats_sql()`, `get_thermal_indices()` . . . 17
6.4 Thermal stress classification . . . 17
6.5 Nature-Based Solutions coverage — `calculate_nbs_coverage()` . . . 18

7 Map editing and what-if backup (mainMap app) **18**
7.1 Editable-layer allowlist — `EDITABLE_LAYERS`, `is_editable()` . . . 18
7.2 Attribute and geometry form — `_build_form()`, `feature_form()` . . . 19
7.3 Identifiers for new objects — `_next_negative_pk()` . . . 19
7.4 Deferred-constraint check — `_check_constraints()` . . . 19
7.5 Change journal — `journal()` . . . 19
7.6 Restore — `restore_backup()` . . . 20
7.7 Map-side editor — `Editing.js` . . . 20

8 Imported climate-effect indicators (importer app, Klimaateffectatlas) 21
8.1 Value transforms — `_prepare_features()` 21
8.2 Join and write rules — `_keep_existing()`, `_enrich_rows()` 21

9 Summary table 21

1 Scope and conventions

Each entry below documents one function or model `save()` method from the codebase. Entries are grouped by Django app, matching the project's domain decomposition. For every entry we record:

- **Location** — file and line number in the repository at time of writing.
- **Equation** — the exact arithmetic the code performs, in mathematical notation.
- **Variables** — symbol, meaning, unit, and source field/model.
- **DAG edge(s)** — the causal edge(s) in `core/DAG.dot` this computation is claimed to implement.
- **Status** — whether the DAG edge is genuinely derived from its source field (implemented), derived but with a known caveat or partial dependency (partial), or claimed in a docstring without being computed (gap).

Units follow the project convention: population counts are dimensionless integers; areas are km^2 unless noted; water volumes are m^3 or Mm^3 (explicitly distinguished per entry, since several models mix the two); monetary values are EUR.

2 Administrative app — population projection

Source: `administrative/population.py`, `administrative/models.py`. Population data originates from CBS table 85173NED (*Regionale prognose 2023–2050*), published per *gemeente* (city) [1].

2.1 Compound annual growth rate — `extrapolate()`

Location: `administrative/population.py:67`

Projects a city's population for a year outside the imported CBS horizon, using the compound annual growth rate implied by the earliest and latest imported years.

$$r = \left(\frac{v_1}{v_0}\right)^{\frac{1}{y_1 - y_0}} - 1 \tag{1}$$

$$P(y) = v_a \, (1 + r)^{(y - y_a)} \tag{2}$$

Variables:

- y_0, y_1 — earliest / latest year with an imported `PopulationProjection` record for the city
- v_0, v_1 — projected population at y_0, y_1
- r — implied compound annual growth rate
- y_a, v_a — anchor year/value: (y_1, v_1) if $y > y_1$, else (y_0, v_0)

DAG edge: *DAG edge: Population_Growth → Total_Population* — implemented

2.2 City-level growth factor — `_city_growth_factor()`

Location: `administrative/population.py:103`

Ratio of a city's CBS-projected population between a baseline year and a target year, used to scale District/Neighbourhood populations without relying on possibly-incomplete local administrative coverage.

$$f(y) = \frac{P_{\text{city}}(y)}{P_{\text{city}}(y_{\text{baseline}})}, \qquad y_{\text{baseline}} = \begin{cases} 2025 & \text{if imported} \\ \min(y_{\text{imported}}) & \text{otherwise} \end{cases} \tag{3}$$

DAG edge: *DAG edge: Population_Growth → Total_Population* (District/Neighbourhood level) — implemented

2.3 District/Neighbourhood projection — `population_by_year()`

Location: `administrative/population.py:124`

$$P_{\text{unit}}(y) = P_{\text{unit}}^{\text{current}} \times f(y) \tag{4}$$

Applied to the unit's own accurate current population rather than its share of `city.currentPopulation`, because the latter reflects only the districts/neighbourhoods imported locally, not CBS's whole-city total (see docstring, `administrative/population.py:11--22`).

Fallback (fewer than two imported years for the city): share-of-city approach,

$$P_{\text{unit}}(y) = \frac{P_{\text{unit}}^{\text{current}}}{P_{\text{city}}^{\text{current}}} \times P_{\text{city}}(y) \tag{5}$$

DAG edge: *DAG edge: Population_Growth → Total_Population* — implemented

2.4 Province projection

Location: `administrative/population.py:132--138`

$$P_{\text{province}}(y) = \sum_{c \in \text{ cities of province}} P_c(y) \tag{6}$$

DAG edge: *DAG edge: Population_Growth → Total_Population* — implemented

2.5 What-if growth adjustment — `_adjust()`

Location: `administrative/population.py:59`

$$v_{\text{adjusted}} = v \times \left(1 + \frac{g}{100}\right)^{\max(y-2025,\ 0)} \tag{7}$$

where g is `growth_adjust_pct`, a user-supplied what-if slider bounded to $[-5, +5]$ (`population_params()`, `administrative/population.py:205`).

2.6 City annual growth rate — `annual_growth_rate()`

Location: `administrative/population.py:90` (as of the `City.popGrowthRate` implementation)

$$\texttt{popGrowthRate} = \left(\left(\frac{v_1}{v_0} \right)^{\frac{1}{y_1 - y_0}} - 1 \right) \times 100 \tag{8}$$

Same CAGR as `extrapolate()`, expressed in percent per year and written onto `City.popGrowthRate` in `City.save()` (`administrative/models.py`) whenever at least two `PopulationProjection` years exist for the city.

DAG edge: *DAG edge: Population_Growth → Total_Population* (`City.popGrowthRate`) — implemented

2.7 Population and density cascade — `City.save()`, `Province.save()`

Location: `administrative/models.py`

$$P_{\text{city}} = \sum_{d \in \text{districts of city}} P_d \quad \text{(only if districts report a nonzero total)} \tag{9}$$

$$P_{\text{province}} = \sum_{c \in \text{cities of province}} P_c \tag{10}$$

$$\rho = \frac{P}{A}, \qquad A = \frac{\text{geom.area}}{10^6} \ \text{km}^2 \tag{11}$$

Density ρ in people/km^2. The Neighbourhood → District → City → Province cascade is signal-driven (`administrative/signals.py`), using `QuerySet.update()` rather than `.save()` to avoid re-triggering `post_save` recursively.

DAG edge: (population aggregation, not a numbered DAG node) — implemented

2.8 Urban area cascade and source computation — `administrative/signals.py`, `physicalEnv/signals.py`

Location: `administrative/signals.py` (cascade), `physicalEnv/signals.py` (source)

$$A^{\text{urban}}_{\text{nbhd}} = \sum_{v \in \text{qualifying LandCoverVector, latest year}} \left| \text{geom}_{\text{nbhd}} \cap \text{geom}_v \right| / 10^6 \ \text{km}^2 \tag{12}$$

$$A^{\text{urban}}_{\text{district}} = \sum_{n \in \text{neighbourhoods of district}} A^{\text{urban}}_n \tag{13}$$

$$A^{\text{urban}}_{\text{city}} = \sum_{d \in \text{districts of city}} A^{\text{urban}}_d \tag{14}$$

$$A^{\text{urban}}_{\text{province}} = \sum_{c \in \text{cities of province}} A^{\text{urban}}_c \tag{15}$$

A `physicalEnv.LandCoverVector` polygon qualifies if its `land_cover_type.class_name` case-insensitively contains any of *urban fabric*, *road*, *rail*, *transport*, restricted to the most recent `year` present for that city (older survey years are not summed, to avoid double-counting repeat surveys). The intersection area is computed in PostGIS via `ST_Intersection` (Django's `Intersection`/`Area` functions).

Neighbourhood → District → City → Province propagation reuses the same `_recompute_population()` helper as the population cascade above, generalised to sum `urban_area` alongside `currentPopulation` in one aggregate; both use `QuerySet.update()`, not `.save()`. Unlike population, an aggregate with no children reporting a value is left `None` (unknown) rather than coerced to 0. Because a `LandCoverVector` write only fires `post_save` on itself (not on `Neighborhood`), the `physicalEnv` signal re-enters the `administrative` cascade manually after writing each affected Neighbourhood's `urban_area`.

DAG edge: `LandCover -> Urbanization` is the closest existing edge in `core/DAG.dot`, but it runs in the opposite direction of what this computes (LandCover only feeds a not-yet-modelled `Urbanization` node, per `TODO.md`), and no Neighbourhood/District/City/Province/LandCoverVector nodes exist in the DAG at all — partial (cascade and source computation fully implemented; no `Urbanization` model backs a DAG edge for it)

2.9 Urban-fabric buffer around new buildings — `urbanize_around()`, `building_created`

Source: `physicalEnv/landcover.py`, receiver in `physicalEnv/signals.py`. When a building is drawn in the map editor (negative id), the land cover of its city's latest year is rewritten around it:

$$B = \text{buffer}(\text{geom}_{\text{building}},\ 10\ \text{m}) \cap \text{geom}_{\text{city}} \tag{16}$$

$$\mathcal{L}' = \{\, P \setminus B : P \in \mathcal{L},\ P \setminus B \neq \emptyset \,\} \cup \{B \text{ as urban fabric}\} \tag{17}$$

$$\text{percentage}_P = |P|/10^6 \,/\, A_{\text{city}} \times 100 \tag{18}$$

where $\mathcal{L}$ is the set of `LandCoverVector` polygons of the city's latest `year`. The buffer is cut out of every polygon it touches (`ST_Difference`; polygons left empty are deleted) and inserted as one new polygon of class *urban fabric*, so the layer stays free of overlaps and the urban-area cascade above counts each square metre once. Imported (BAG) buildings and later moves or deletes do not trigger it; the editor's backup journal records the rewritten rows, so a restore brings the original land cover back.

DAG edge: *DAG edge: New_Units → LandCover* — partial (map-editor buildings only; `calculate_new_units()` pipeline counts still do not change the land cover)

3 Physical environment app — soil hydrology and infiltration

Source: `physicalEnv/soil.py`, `physicalEnv/hilucs.py`, `physicalEnv/models.py`. Soil polygons come from the Dutch soil map 1:50 000 (*Bodemkaart*, 2025 edition) and groundwater depths from the groundwater-level grids of BIS Nederland [12]; land cover is the CORINE-classified BRT *Bodemgebruik* layer [13]. Together they implement the `LandCover`→`Infiltration` and `Soil_Type`→`Infiltration` edges with the SCS Curve Number method [9, 10, 11]. Everything except the spatial composition query (§3.3) is pure Python, so the dashboard re-evaluates the equations for any rain depth without querying the database.

3.1 Hydrologic soil group — `classify_soil_group()`, `SoilType.save()`

Location: `physicalEnv/soil.py:82`, `physicalEnv/models.py:61`

The soil map names each unit in Dutch with the texture of its layers (e.g. *Koopveengronden op zand*: peat on sand). Each texture word w found in the name n maps to a hydrologic soil group $g(w)$, and the most restrictive layer sets the group, since the least transmissive layer limits infiltration [10]:

$$G(n) = \max_{\prec}\{\, g(w) : w \in \text{words}(n) \,\}, \qquad \text{A} \prec \text{B} \prec \text{C} \prec \text{D} \tag{19}$$

with $G(n)$ undefined when the name contains no texture word. The phrase *geen zand* ("no sand") is removed before matching.

Group	Dutch texture words (soil map)	Texture
A	*zand*, *grind* (incl. loamy and clayey sand)	sand, gravel
B	*zavel* (8–25 % lutum), *leem* (not *leemarm*, *lemig*, *keileem*)	loam, silt loam
C	*lichte klei*; *knip* (dense knip-clay horizon)	clay loam
D	*klei*, *zware klei*, *veen*, *moerig*, *slik*, *drecht/lied* (clay on peat), *keileem*, *potklei*, *kleefaarde*	clay, peat

All 171 distinct soil names of the 2025 map receive a group (A: 46 names, B: 33, C: 16, D: 76). `SoilType.save()` stores $G(n)$ and the final infiltration rate range $[f_{\min}, f_{\max}]$ of the group [10]:

Group	Final infiltration rate f (mm/h)	Runoff potential
A	> 7.6	low
B	3.8–7.6	moderate
C	1.3–3.8	moderately high
D	0–1.3	high

DAG edge: *DAG edge: Soil_Type → Infiltration* — implemented (together with §3.3–3.5)

3.2 Curve number — `curve_number()`

Location: `physicalEnv/soil.py:138`

The curve number $\mathrm{CN}(c, G)$ of a land-cover class c on soil group G is read from the TR-55 tables for antecedent moisture condition II [9]. CORINE class names are matched by keyword at a word start (so *road* does not match *broad-leaved forest*); the first matching cover type wins:

CORINE class (keyword)	TR-55 cover type	A	B	C	D
water courses/bodies, sea, lagoons, estuaries	open water	100	100	100	100
marshes, peat bogs, salines, intertidal flats	saturated wetland (impervious)	98	98	98	98
green urban areas, sport and leisure	open space, good condition	39	61	74	80
discontinuous urban fabric; urban fabric	residential, $\leq 1/8$ acre (65 % imperv.)	77	85	90	92
continuous urban fabric	commercial and business (85 % imperv.)	89	92	94	95
industrial/commercial units, ports, airports	industrial (72 % imperv.)	81	88	91	93
road and rail networks	paved roads, open ditches, incl. right-of-way	83	89	92	93
mineral extraction, dumps, construction	newly graded areas	77	86	91	94
arable land, irrigated land, rice	row crops, straight row, good	67	78	85	89
vineyards, fruit trees, olive groves	woods–grass combination (orchard), good	32	58	72	79
natural grasslands	meadow	30	58	71	78
pastures	pasture, good	39	61	74	80
complex cultivation, agro-forestry	small grain, straight row, good	63	75	83	87
forests (broad-leaved, coniferous, mixed)	woods, good	30	55	70	77
moors and heathland	brush, good	30	48	65	73
sclerophyllous vegetation, transitional woodland	brush, fair	35	56	70	77
beaches, dunes, bare rock, sparse vegetation, burnt	fallow, bare soil	77	86	91	94

Imperviousness of built-up land enters through these curve numbers; a class without a match (e.g. glaciers) or a piece without a soil group is left out and reported as unclassified.

3.3 Shallow groundwater and drained land — `soil_landcover_composition()`

Location: `physicalEnv/soil.py:206`; constants `SHALLOW_GROUNDWATER_CM` (`:30`), `SEASONS` (`:34`), `DRAINED_LAND_COVER` (`:48`)

A soil whose water table lies within $h^* = 60\,\text{cm}$ of the surface belongs to group D whatever its texture; NEH 630 gives such soils a dual group (A/D, B/D, C/D), the texture letter applying where the land is drained [10]. The query intersects the unit u with the soil map and the most recent land-cover year, and splits every piece p by the area where the season's groundwater depth h_s is

shallower than h^*:

$$S_s = \bigcup \{ \text{ cells } x \subset u : h_s(x) < h^* \} \tag{20}$$

$$G'(p) = \begin{cases} \mathrm{D} & \text{on } p \cap S_s \text{ if } c(p) \notin \mathcal{D} \\ G(p) & \text{otherwise} \end{cases} \tag{21}$$

where $s \in \{\text{wet}, \text{dry}\}$ selects $h_{\text{wet}} = \text{GHG}$ (mean highest groundwater level) or $h_{\text{dry}} = \text{GLG}$ (mean lowest), and $\mathcal{D}$ is the set of drained land covers (urban fabric, industrial/commercial units, road and rail networks, port areas, airports). GHG and GLG are 50 m grids in cm below ground level, capped at 254 ("254 cm or deeper"); cells without a value (255) are not shallow. BIS Nederland leaves most built-up cells (85 % in Enschede) and water bodies without a value, consistent with drained, impervious land; water and wetlands need no rule since their curve number is independent of the soil group. The cells are reclassified to shallow/deep and dissolved into polygons (`ST_Reclass`, `ST_DumpAsPolygons`, `ST_Union`) before the intersection, so each piece is split once rather than per cell. The function returns the area of every (G', c) combination, the texture-only group areas, $|S_s \cap \bigcup_{c \notin \mathcal{D}} p|$ (moved to D) and $|S_s \cap \bigcup_{c \in \mathcal{D}} p|$ (kept as drained).

3.4 SCS runoff and infiltration coefficient — `scs_runoff_mm()`, `infiltration_coefficient()`

Location: `physicalEnv/soil.py:149, 165`

$$S = \frac{25400}{\mathrm{CN}} - 254, \qquad I_a = 0.2\,S \tag{22}$$

$$Q = \begin{cases} \dfrac{(P - I_a)^2}{P - I_a + S} & P > I_a \\ 0 & P \le I_a \end{cases} \tag{23}$$

$$C_{\text{inf}} = \frac{P - Q}{P} \tag{24}$$

where P is the rain depth of one event (mm), S the potential maximum retention (mm), I_a the initial abstraction (mm), Q the direct runoff (mm) and C_{inf} the infiltration coefficient, the share of the rain that does not run off [11]. $\mathrm{CN} = 100$ gives $Q = P$.

3.5 Area-weighted unit summary — `summarize_infiltration()`

Location: `physicalEnv/soil.py:172`

For the pieces i of a unit, with area A_i and curve number $\mathrm{CN}_i = \mathrm{CN}(c_i, G'_i)$:

$$V_{\text{inf}} = \sum_i A_i \, \frac{P - Q(P, \mathrm{CN}_i)}{1000}, \qquad V_{\text{run}} = \sum_i A_i \, \frac{Q(P, \mathrm{CN}_i)}{1000} \quad [\mathrm{m}^3] \tag{25}$$

$$C_{\text{inf},u} = \frac{V_{\text{inf}}}{V_{\text{inf}} + V_{\text{run}}}, \qquad \overline{\mathrm{CN}} = \frac{\sum_i A_i \, \mathrm{CN}_i}{\sum_i A_i} \tag{26}$$

Runoff is computed per piece before weighting: since Q is convex in CN, evaluating $Q(P, \overline{\mathrm{CN}})$ would overstate infiltration. $\overline{\mathrm{CN}}$ is reported for reference only.

3.6 HILUCS land-use codes — `parse_hilucs_href()`, `hilucs_q()`

Location: `physicalEnv/hilucs.py:18, 48`

Zoning elements of the INSPIRE Planned Land Use feed reference their land use as a URI of the HILUCS codelist [14], e.g. `.../HILUCSValue/6_3_2_WaterAreasNotInOtherEconomicUse`. The code is read from the URI's leading digits (`6_3_2` $\mapsto$ 6.3.2) and matched against `HILUCSLandUse.code` (98 registry values, loaded by migration); the full URI is never used, since the PDOK file misspells some (`5_2_...Comptible...`). A code k selects itself and every code below it in the hierarchy:

$$\mathrm{sel}(k) = \{\, k' : k' = k \ \vee \ k' \text{ starts with } k. \,\} \tag{27}$$

4 Water supply app

Source: `watersupply/calculations.py`, `watersupply/models.py`, `watersupply/signals.py`.

4.1 Per-capita consumption — `_get_consumption_capita()`

Location: `watersupply/calculations.py:16`

$$\bar{c} = \mathrm{avg}_{\,c \,\in\, \mathrm{cities}(u)} \big(\texttt{consumption_capita_L_d}\big), \qquad [\bar{c}] = \mathrm{L/person/day} \tag{28}$$

4.2 Total water demand — `calculate_total_demand()`, `TotalWaterDemand.save()`

Location: `watersupply/calculations.py:27`; `watersupply/models.py:70`

$$D_{\mathrm{day}} = \frac{\bar{c}}{1000} \times P \qquad [\mathrm{m^3/day}] \tag{29}$$

$$\texttt{demandDay} = \frac{P \times \bar{c}}{10^9} \qquad [\mathrm{Mm^3/day}] \tag{30}$$

$$\texttt{demandYR} = \texttt{demandDay} \times 365 \tag{31}$$

where P is `city.currentPopulation`. Note the two representations differ by a factor of 10^6 ($\mathrm{m^3}$ vs. $\mathrm{Mm^3}$); `calculate_supply_security()` converts explicitly (§4.6).

DAG edges: *DAG edge: Total_Population → Total_Water_Demand*, *DAG edge: Consumption-Capita → Total_Water_Demand* — implemented

4.3 Total consumption — `ConsumptionCapita.save()`

Location: `watersupply/models.py:38`

$$\texttt{total_consumption_m3_yr} = \frac{\texttt{consumption_capita_L_d}}{1000} \times P \times 365 \tag{32}$$

DAG edges: *DAG edge: Total_Population → Total_Consumption*, *DAG edge: ConsumptionCapita → Total_Consumption* — implemented

4.4 Total extraction — `calculate_total_extraction()`

Location: `watersupply/calculations.py:38`

$$E = \left(\sum_{w \,\in\, \text{active wells} \,\cap\, \mathrm{geom}(u)} q_w \right) \times 86400, \qquad [q_w] = \mathrm{m^3/s}, \ [E] = \mathrm{m^3/day} \tag{33}$$

DAG edge: *DAG edge: Available_FW → Total_Extraction* — implemented

4.5 Total production — `calculate_total_production_day()`

Location: `watersupply/calculations.py:51`

$$\Pi = E + I, \qquad I = \sum_{\text{active imports}} \texttt{quantity_m3_d} \tag{34}$$

DAG edges: *DAG edge: Total_Extraction → Total_Water_Prod*, *DAG edge: Imported_Water → Total_Water_Prod* — implemented

4.6 Supply security & service time

Location: `calculate_supply_security()` (`watersupply/calculations.py:73`); `SupplySecurity.save()` (`watersupply/models.py:107`)

$$D_{\mathrm{m^3/day}} = \texttt{demandDay} \times 10^6 \tag{35}$$

$$S = \frac{\Pi}{D_{\mathrm{m^3/day}}} \qquad \text{(ratio; } \times 100 \text{ for \%)} \tag{36}$$

$$T_{\text{service}} = \begin{cases} 24 & \Pi \geq D_{\mathrm{m^3/day}} \\ 24 \times \dfrac{\Pi}{D_{\mathrm{m^3/day}}} & \Pi < D_{\mathrm{m^3/day}} \end{cases} \qquad [\text{h/day}] \tag{37}$$

DAG edges: *DAG edge: Total_Water_Demand → Supply_Security*, *DAG edge: Total_Water_Prod → Supply_Security*, *DAG edge: Supply_Security → Service_Time* — implemented

4.7 Energy consumption — `calculate_energy_consumption()`

Location: `watersupply/calculations.py:100`

$$K = \sum_{w} \left(\texttt{pumpEnergyRate_kWh_h}_w \times \texttt{OperationTime_h_day}_w\right) + 1000 \times \sum \texttt{EnergyConsumption_MW_day} \tag{38}$$

$[K] = \text{kWh/day}$. **DAG edges:** *DAG edge: Total_Extraction → Energy_Consumption*, *DAG edge: Total_Water_Prod → Energy_Consumption*, *DAG edge: Energy_Cost → Energy_Consumption* — partial (Energy_Cost side of the edge is not read back into this function)

4.8 Pump emissions — `ExtractionWater.save()`

Location: `watersupply/models.py:317`

$$R_{\mathrm{CO_2}} = \texttt{pumpEnergyRate_kWh_h} \times \texttt{pumpEmmissionFactor_kg_CO2_kWh} \tag{39}$$

$$M_{\text{day}} = R_{\mathrm{CO_2}} \times \texttt{OperationTime_h_day}, \qquad M_{\text{yr}} = M_{\text{day}} \times 365 \tag{40}$$

DAG edge: *DAG edge: Total_Extraction → CO2_Emission* — implemented (per-well; `calculate_co2_emission()` sums across wells)

4.9 Extraction OPEX and environmental cost — `ExtractionWater.save()`

Location: `watersupply/models.py:346--357`

$$\texttt{opex_EUR_m3} = \texttt{labor} + \texttt{energy} + \texttt{chemicals} + \texttt{tax} \quad \text{(only when at least one component is known)} \tag{41}$$

$$\mathrm{CO_2\ cost} = \frac{M_{\text{day}}}{V_{\text{day}}} \times \mathrm{price}_{\mathrm{CO_2}}, \qquad V_{\text{day}} = q \times \texttt{OperationTime_h_day} \times 3600 \tag{42}$$

4.10 Consolidated OPEX — `OPEX.save()`

Location: `watersupply/models.py:640`

$$\text{Cost}_{\text{extraction}} = \sum \left(\texttt{opex_EUR_m3} \times \texttt{current_extraction_Mm3_yr} \times 10^6\right) \tag{43}$$

$$\text{Cost}_{\text{imported}} = \sum \left(\texttt{quantity_m3_d} \times 365 \times \texttt{price_EUR_m3}\right) \tag{44}$$

$$\text{Cost}_{\text{treatment}} = \overline{\texttt{UnitaryOPEX_EUR_m3}}_{\text{year}} \times V_{\text{yr}} \tag{45}$$

$$\text{Cost}_{\text{network}} = \sum \left(\texttt{maitenanceCost_EUR_km} \times \texttt{length_km}\right) \tag{46}$$

$$\texttt{totalOPEX_EUR} = \text{Cost}_{\text{extraction}} + \text{Cost}_{\text{imported}} + \text{Cost}_{\text{treatment}} + \text{Cost}_{\text{network}} \tag{47}$$

$$\texttt{UnitaryOPEX_EUR_m3} = \frac{\texttt{totalOPEX_EUR}}{V_{\text{yr}}}, \qquad V_{\text{yr}} = V_{\text{extraction}} + V_{\text{imported}} \tag{48}$$

`TotalWaterProduction`'s own energy cost is deliberately excluded, since it is already embedded in `opex_EUR_m3` (see inline comment, `watersupply/models.py`).

$$\texttt{OPEX_recovered_PCT} = \frac{\sum \texttt{Recovery_EUR}}{\texttt{totalOPEX_EUR}} \times 100 \tag{49}$$

DAG edges: *DAG edge: Total_Extraction/Total_Water_Prod → OPEX*, *DAG edge: Imported_Water → OPEX*, *DAG edge: WT_Cost → OPEX*, *DAG edge: Network → OPEX*, *DAG edge: Metered_Res_Water → OPEX_Recovery* — implemented

4.11 OPEX recovery (dashboard) — `calculate_opex_recovery()`

Location: `watersupply/calculations.py:205`

$$\text{Recovery}\% = \frac{\sum_{\text{metered users}} \texttt{Recovery_EUR}}{\texttt{totalOPEX_EUR}_{\text{year}}} \times 100 \tag{50}$$

DAG edges: *DAG edge: OPEX → OPEX_Recovery*, *DAG edge: CollectionRatio → OPEX_Recovery*, *DAG edge: NRW → OPEX_Recovery* — partial (computes directly from stored totals; does not call `calculate_collection_ratio()` or reference NRW, despite the docstring)

4.12 Billed consumption and revenue — `MeteredResidential.save()`

Location: `watersupply/models.py:172`

$$\gamma = \frac{\texttt{collected_meters}}{\texttt{installed_meters}} \tag{51}$$

$$V_{\text{yr}} = \frac{\texttt{consumption_capita_L_d}}{1000} \times 365 \times \texttt{populationServed} \times \gamma \tag{52}$$

$$\texttt{Recovery_EUR} = V_{\text{yr}} \times \texttt{userTariff_EUR_m3} \tag{53}$$

Uses the city's most recent `ConsumptionCapita` record (not filtered by year), applied to local `populationServed` rather than the whole-city `total_consumption_m3_yr`.

4.13 Collection ratio — `calculate_collection_ratio()`

Location: `watersupply/calculations.py:184`

$$\text{CR}\% = \frac{\sum \texttt{collected_meters}}{\sum \texttt{installed_meters}} \times 100 \tag{54}$$

DAG edges: *DAG edge: Metered_Res_Water → CollectionRatio*, *DAG edge: User_Acceptance_WS → CollectionRatio*, *DAG edge: Water_Tariff_Afford → CollectionRatio* — partial (ignores `userAffordability_PCT` and acceptance rate despite the docstring)

4.14 Coverage — `calculate_coverage()`, `CoverageWaterSupply.save()`

Location: `watersupply/calculations.py:239`; `watersupply/models.py:448`

$$G_{\text{buffer}} = \bigcup_{p \in \text{pipes}(c)} \text{buffer}(p,\ 500\,\text{m}) \tag{55}$$

$$A_{\text{covered}} = \frac{\text{area}(G_{\text{buffer}})}{10^6}\ [\text{km}^2] \tag{56}$$

$$\text{Coverage}\% = \frac{H_{\text{covered}}}{H_{\text{total}}} \times 100 \tag{57}$$

where H_{covered} counts households in `UsersLocation` points spatially within G_{buffer}, and H_{total} sums `usersTotal` across the city's neighbourhoods. **DAG edges:** *DAG edge: Network → Coverage_WS_Area*, *DAG edge: CityArea → Coverage_WS_Area*, *DAG edge: NumberUsers → Coverage_WS* — implemented

4.15 Non-Revenue Water and ILI

Location: `calculate_nrw()` (`watersupply/calculations.py:275`); `NonRevenueWater.save()` (`watersupply/models.py:551`)

$$\text{NRW}_{\text{apparent}} = \sum_{\text{type}=A} \texttt{loss_Quantity_m3} \tag{58}$$

$$\text{NRW}_{\text{real}} = \sum_{\text{type}=R} \texttt{loss_Quantity_m3} \tag{59}$$

$$\text{CARL} = \sum_{\text{type}=R,\ \text{year}} \texttt{loss_Quantity_m3} \tag{60}$$

$$\text{UARL} = \sum_{\text{type}=R,\ \text{year}} \texttt{loss_Quantity_m3} \times \frac{\texttt{UnavoidableLossses_PCT}}{100} \tag{61}$$

$$\text{ILI} = \frac{\text{CARL}}{\text{UARL}} \tag{62}$$

The Infrastructure Leakage Index formulation follows the IWA Water Loss Task Force methodology [3]. **DAG edges:** *DAG edge: Real_Losses → NRW, ILI*, *DAG edge: Apparent_Losses → NRW, ILI* — partial (ILI is only defined for Real losses; *DAG edge: Apparent_Losses → ILI* is unimplemented; `NonRevenueWater` records are entered directly rather than linked to `PipeNetwork`, so *DAG edge: Network → Real_Losses* is a gap)

4.16 Water quality — `calculate_water_quality()`

Location: `watersupply/calculations.py:144`

$$\text{Compliance}\% = \frac{\sum \texttt{samplesWaterQuality_OK}}{\sum \texttt{samplesWaterQualityTaken}} \times 100 \tag{63}$$

DAG edges: *DAG edge: WT_Efficiency → Samples_WQ*, *DAG edge: Samples_Taken → Samples_WQ*, *DAG edge: Samples_WQ → User_Acceptance_WS* — gap (`acceptance_rate` in the return value is avg(`acceptanceRate`), an independent stored field unrelated to the computed compliance percentage)

4.17 Available fresh water and drought area

Location: `watersupply/calculations.py:308, 323`

$$\text{AFW} = \sum_{\text{intersecting } u} \texttt{totalQuantity_Mm3} \tag{64}$$

$$A_{\text{drought}} = \sum_{\text{intersecting } u,\ \text{year}} \texttt{areaAffected_km2} \tag{65}$$

DAG edges: *DAG edge: Infiltration → Available_FW*, *DAG edge: Meteorology → Available_FW*, *DAG edge: Total_Extraction → Area_Drought* — gap for the Infiltration side: infiltration is now computed per event (§4.18), but `AvailableFreshWater.infiltrationRate_cm_h` is still a flat, manually entered field — turning the event coefficient into a recharge volume is open

4.18 Infiltration indicator (dashboard) — `calculate_infiltration()`, `_infiltration_indicator`

Location: `watersupply/calculations.py:383`, `watersupply/views.py:177`

The water dashboard's Infiltration card evaluates §3 for the administrative unit u with two what-if controls: the *Rain Event* slider sets $P \in [1, 100]$ mm (default 25; KNMI's daily classes mark *heavy* from 25 mm and *extreme* from 50 mm), and the *Groundwater* switch sets the season s (wet = GHG, default; dry = GLG). P enters only the pure functions and the spatial composition depends on s alone, so the composition could be cached per season; the current implementation recomputes it on every request, together with the rest of `_get_adminUnit_data()`. Displayed: $C_{\text{inf},u}$ (%), $\overline{\text{CN}}$, V_{inf} and V_{run} (m^3), the classified and unclassified areas, the share of the classified area moved to D by shallow groundwater and the share kept as drained, and the texture-only soil-group shares with their rate ranges $[f_{\min}, f_{\max}]$.

DAG edges: *DAG edge: LandCover → Infiltration*, *DAG edge: Soil_Type → Infiltration* — implemented

4.19 Pipe geometry — `PipeNetwork.save()`

Location: `watersupply/models.py:420`

$$L_{\text{km}} = \frac{\text{length}(\texttt{geom})}{1000}, \qquad d_{\text{in}} = \frac{d_{\text{mm}}}{25.4} \tag{66}$$

5 Housing app

Source: `housing/calculations.py`, `housing/models.py`.

5.1 Supply/demand deficit

Location: `calculate_supply_demand()` (`housing/calculations.py:14`), `calculate_supply_demand_for_unit(` (`:43`)

$$\text{Deficit} = D - S \tag{67}$$

$$\text{Deficit}\% = \frac{D - S}{D} \times 100 \tag{68}$$

where $S =$ `supply_units`, $D =$ `demand_units` (stored, not derived from population). **DAG edges:** *DAG edge: Urbanization → Housing_Demand*, *DAG edge: New_Units → Housing_Supply*, *DAG edge: Housing_Demand → House_Price_Index* — gap (`demand_units` is a stored value, not derived from §2's population projection; no `Urbanization` model exists)

5.2 Pipeline units — `calculate_new_units()`

Location: `housing/calculations.py:66`

$$U_{\text{completed}} = \sum_{\texttt{year_expected_completion} \le y} \texttt{project_units} \tag{69}$$

$$U_{\text{pipeline}} = \sum_{\texttt{year_expected_completion} > y} \texttt{project_units} \tag{70}$$

DAG edges: *DAG edge: Zoning_Status → New_Units*, *DAG edge: Network → New_Units*, *DAG edge: New_Units → Housing_Supply*, *DAG edge: New_Units → LandCover* — partial (counts completions/pipeline; never writes back to `LandCover` or reads `Zoning_Status`/`Network` as inputs — the land-cover side of the edge is implemented for buildings drawn in the map editor, see `urbanize_around()`)

5.3 Mortgage indicators — `calculate_mortgage_indicators()`

Location: `housing/calculations.py:123`

$$\overline{m} = \text{avg}(\texttt{monthlyPayment}), \quad \overline{\imath} = \text{avg}(\texttt{interestRate}), \quad \overline{L} = \text{avg}(\texttt{totalLoanAmount}) \tag{71}$$

plus province-level policy fields (`interest_rate`, `LTV_limit`, `LTI_limit`, `mortgageRate`) read as-is. **DAG edges:** *DAG edge: Central_Bank → Mortgage*, *DAG edge: Credit_Supply → Mortgage*, *DAG edge: Mortgage → Affordability_Stress*, *DAG edge: Mortgage → Income_Expenses* — gap for the two outgoing edges (averages are exposed on the dashboard but never fed back into `HousingAffordability`)

5.4 House Price Index — `calculate_house_price_index()`

Location: `housing/calculations.py:200`

$$\Delta_{\text{YoY}}\% = \frac{\overline{\text{HPI}}_y - \overline{\text{HPI}}_{y-1}}{\overline{\text{HPI}}_{y-1}} \times 100 \tag{72}$$

DAG edges: *DAG edge: Housing_Demand → House_Price_Index*, *DAG edge: Property → House_Price_Index* — gap (`HousePriceIndex` has no `save()`; `index_value` is a flat stored field, and this function only averages it)

5.5 Affordability index — `HousingAffordability.save()`

Location: `housing/models.py:150`

$$\mathrm{AI} = \frac{\texttt{medianHousePrice}}{\texttt{medianIncome}} \tag{73}$$

$$\mathrm{Income}_{\mathrm{disposable}} = \texttt{medianIncome} - \texttt{medianExpenditure} \tag{74}$$

The house-price-to-income ratio formulation is the standard affordability metric used in comparative housing-affordability studies (e.g. Demographia's Median Multiple) [4]. **DAG edges:** *DAG edge: Property → Affordability_Stress*, *DAG edge: Income_Expenses → Affordability_Stress*, *DAG edge: Mortgage → Affordability_Stress*, *DAG edge: Water_Tariff_Afford → Income_Expenses*, *DAG edge: Rent → Income_Expenses*, *DAG edge: Mortgage → Income_Expenses* — gap (computed only from this model's own stored fields; never reads `Mortgage`, `Rentals`, or watersupply tariff data — an open TODO in `housing/models.py` also questions whether disposable income should replace median income in the numerator's denominator)

5.6 Zoning composition — `calculate_zoning()`

Location: `housing/calculations.py:297`; groups `ZONING_GROUPS` (`:289`)

$$\mathrm{Area}\%_t = \frac{\sum_{z\,:\,\texttt{zone_type}(z)\in K_t} \texttt{area}(z)}{\sum_{\text{all zones}} \texttt{area}(z)} \times 100 \tag{75}$$

`ZoningArea.zone_type` is a foreign key to the HILUCS codelist (§3), filled from each INSPIRE zoning element's `hilucsLandUse` reference. The four dashboard groups are code sets with all their sub-codes (`hilucs_q()`): $K_{\mathrm{residential}} = \mathrm{sel}(5) \setminus \mathrm{sel}(5.2)$, $K_{\mathrm{commercial}} = \mathrm{sel}(3.1) \cup \mathrm{sel}(3.2)$, $K_{\mathrm{industrial}} = \mathrm{sel}(2)$, $K_{\mathrm{mixed}} = \mathrm{sel}(5.2)$. Other land uses (e.g. water, agriculture) count in the denominator only. **DAG edges:** *DAG edge: Housing_Demand → Zoning_Status*, *DAG edge: Zoning_Status → Property*, *DAG edge: Zoning_Status → New_Units* — gap (descriptive aggregation only; no model consumes `Zoning_Status` as an input)

5.7 Property market and vacancy — `calculate_property_indicators()`

Location: `housing/calculations.py:236`

$$\overline{p} = \mathrm{avg}(\texttt{salePrice_EUR}), \qquad \overline{p}_{\mathrm{EUR/m^2}} = \mathrm{avg}(\texttt{unitaryPrice_EUR_per_sqm}), \qquad \overline{v} = \mathrm{avg}(\texttt{vacancyRate}) \tag{76}$$

DAG edges: *DAG edge: Housing_Supply → Property*, *DAG edge: Zoning_Status → Property*, *DAG edge: Accessibility → Property*, *DAG edge: Property → Affordability_Stress*, *DAG edge: Property → Rent*, *DAG edge: Property → House_Price_Index*, *DAG edge: Property → Mortgage* — gap (no `Accessibility` model exists; none of the outgoing edges are computed — this function only averages stored fields)

6 Urban heat app

Source: `urban_heat/calculations.py`. Thermal-comfort rasters (SVF, T_{mrt}, PET, UTCI) are expected to arrive pre-computed from an external SOLWEIG run [5]; CrossTwin currently only reads and classifies them, it does not derive one raster from another.

6.1 Green area and vegetation coverage

Location: `calculate_green_area()` (`urban_heat/calculations.py:18`), `calculate_vegetation_coverage()` (`:55`)

$$A_{\text{green,lc}} = A_{\text{unit}} \times \frac{\sum \texttt{percentage}_{\text{green classes}}}{100} \tag{77}$$

$$A_{\text{green,total}} = A_{\text{parks}} + A_{\text{green,lc}} \tag{78}$$

$$\text{VegCov\%} = \frac{A_{\text{green,total}}}{A_{\text{unit}}} \times 100 \tag{79}$$

`percentage` is stored relative to the *Province* total area regardless of the queried unit (documented limitation, see `TODO.md`). **DAG edges:** *DAG edge: LandCover → Green_Area*, *DAG edge: Green_Area → Vegetation_Coverage* — partial; *DAG edge: Vegetation_Coverage → DSM* — gap

6.2 Urban morphology / canyon aspect proxy — `calculate_urban_morphology()`

Location: `urban_heat/calculations.py:75`

$$\text{Aspect ratio} \equiv \frac{H}{W} = \frac{\overline{\texttt{height_m}}_{\text{buildings}}}{\overline{\texttt{width}}_{\text{streets}}} \tag{80}$$

The height-to-width (H/W) canyon aspect ratio is the standard urban-canyon geometry descriptor used in urban climatology [6]. **DAG edges:** *DAG edge: Buildings → DSM, Canyon_Aspect*, *DAG edge: Streets → Canyon_Aspect* — partial (H/W is computed as a scalar proxy over the whole admin unit, not written back onto a `Canyon_Aspect` or `DSM` model)

6.3 Raster statistics — `_raster_stats_sql()`, `get_thermal_indices()`

Location: `urban_heat/calculations.py:112, 145`

$$\{\min, \max, \mu, \sigma\} = \text{ST_SummaryStats}\left(\text{ST_Clip}(R,\ \texttt{geom}(u))\right) \tag{81}$$

computed via PostGIS raster functions over the most recently dated raster of each type (LST, T_{mrt}, PET, UTCI, WBGT, SVF, SUHII) intersecting the admin unit. **DAG edges:** *DAG edge: Meteorology → Tmrt, PET, UTCI*, *DAG edge: DSM → SVF → Tmrt, PET*, *DAG edge: LST → PET*, *DAG edge: Tmrt → PET, UTCI* — gap (statistics *over* existing rasters only; no raster is derived from another)

6.4 Thermal stress classification

Location: `classify_pet()` (`:201`), `classify_utci()` (`:225`), `classify_wbgt()` (`:267`)

PET thresholds (in °C) follow the physiologically-based 9-class scheme of Matzarakis and Mayer

[7]:

$$\text{class(PET)} = \begin{cases} \text{Very Cold Stress} & \text{PET} < 4 \\ \text{Cold Stress} & 4 \le \text{PET} < 8 \\ \text{Slight Cold Stress} & 8 \le \text{PET} < 13 \\ \text{No Thermal Stress (cool)} & 13 \le \text{PET} < 18 \\ \text{Comfortable} & 18 \le \text{PET} < 23 \\ \text{No Thermal Stress (warm)} & 23 \le \text{PET} < 29 \\ \text{Slight Heat Stress} & 29 \le \text{PET} < 35 \\ \text{Moderate Heat Stress} & 35 \le \text{PET} < 41 \\ \text{Extreme Heat Stress} & \text{PET} \ge 41 \end{cases} \tag{82}$$

UTCI thresholds follow the 10-class scheme defined by the COST Action 730 UTCI framework [8]:

$$\text{class(UTCI)} = \begin{cases} \text{Extreme Cold Stress} & \text{UTCI} < -40 \\ \vdots & \vdots \\ \text{Extreme Heat Stress} & \text{UTCI} \ge 46 \end{cases} \quad \text{(9 intermediate bands; see } \texttt{urban_heat/calculations.py} \tag{83}$$

WBGT bands (°C: 25/28/30/33 thresholds) are marked **provisional** in the source (`urban_heat/calculations.py` pending confirmation against a named standard.

6.5 Nature-Based Solutions coverage — `calculate_nbs_coverage()`

Location: `urban_heat/calculations.py:283`

$$A_{\text{NBS}} = \sum_{\text{NBS polygons} \cap \texttt{geom}(u)} \texttt{area} \tag{84}$$

DAG edges: *DAG edge: UTCI → NBS*, *DAG edge: PET → NBS* — gap (counts NBS geometries only; never reads PET or UTCI values to relate NBS placement to thermal stress)

7 Map editing and what-if backup (mainMap app)

Source: `mainMap/editing.py`, `mainMap/models.py`, `core/static/js/Editing.js`; design in `docs/MAP_EDITING.m`
Staff users edit built-up objects (`Building`, `Street`, `Park`, `Facility`, `ZoningArea`) directly on the map, and can set one restore point so what-if edits can be undone. These functions implement no DPSIR edge themselves; they are *Response*-side tooling that changes the State variables (buildings, streets, zoning) from which the indicator functions above are computed. Every write goes through the model's `save()`, so the derivations documented elsewhere in this registry (e.g. `Building.save()`, the cache invalidation in `core/signals.py`) apply unchanged.

7.1 Editable-layer allowlist — `EDITABLE_LAYERS`, `is_editable()`

Location: `mainMap/editing.py:36`, `:55`

For a layer ℓ and user u,

$$\text{editable}(\ell, u) = [\, \ell \in \mathcal{E} \,] \wedge [\, u.\texttt{is_staff} \,], \tag{85}$$

where $\mathcal{E}$ is the allowlist. Each entry lists the fields the model derives itself (`exclude`, kept out of the form) and the fields the form must not require (`optional`). For `Building`: `exclude` = {`area_sqm`, `neighborhood`, `last_updated`, `connectivity`}, `optional` = {`buildingType`}; for `ZoningArea`: `exclude` = {`area`}. The layer catalog (`available_layers`) adds `editable` and `edit_geom_type` to an entry only when editable(ℓ, u) holds.

7.2 Attribute and geometry form — `_build_form()`, `feature_form()`

Location: `mainMap/editing.py:71`, `:134`

The browser sends the edited shape as WGS84 GeoJSON in a hidden form field. Django's GIS form field parses it, rejects a geometry whose type differs from the model field's (e.g. a `Polygon` for a `MultiPolygonField`), and reprojects it to the storage CRS:

$$g_{\text{stored}} = T_{4326 \to 28992}(g_{\text{GeoJSON}}) \tag{86}$$

The editor (`Editing.js::_writeGeometry()`) wraps single geometries before sending them when the field is a Multi* type: `Polygon`$(c) \mapsto$ `MultiPolygon`$([c])$. A form posted without a new shape sends back the stored EWKT, which already carries SRID = 28992, so no transformation (and no coordinate drift) occurs. For display, the stored geometry is sent at full resolution as $T_{28992 \to 4326}(g_{\text{stored}})$, not the map's copy, which is simplified below zoom 14 (`_geojson_sql()`).

Saving calls `obj.save()`, so for a `Building` the existing derivations run: `area_sqm` = area(g_{stored}) in m^2, `neighborhood` by point-in-polygon of the centroid, and `buildingType` from `usageFunction` (`builtup/models.py`).

7.3 Identifiers for new objects — `_next_negative_pk()`

Location: `mainMap/editing.py:84`

`Building.id` is the BAG identifier, not generated by the database. A user-created building gets

$$\text{id}_{\text{new}} = \min\left(0, \min_{b \in \texttt{Building}} \text{id}_b\right) - 1, \tag{87}$$

so new objects count down from -1, never collide with (positive) BAG ids, and are recognisable as user-made. Models with a database-generated key (`AutoField`) keep it.

7.4 Deferred-constraint check — `_check_constraints()`

Location: `mainMap/editing.py:94`

Django creates foreign keys as `DEFERRABLE INITIALLY DEFERRED`, and the project's spatial FKs use `on_delete=DO_NOTHING`, so a deletion that leaves a dangling reference (a `Property` pointing at a deleted `Building`) would only fail at `COMMIT`. `SET CONSTRAINTS ALL IMMEDIATE` checks all pending references inside the atomic block; a violation raises `IntegrityError`, rolls back just that operation and is reported as a message. Used by `feature_delete()` (`:176`) and `restore_backup()`.

7.5 Change journal — `journal()`

Location: `mainMap/editing.py:216`; models `EditBackup`, `EditBackupRow` (`mainMap/models.py`)

A backup B is created empty at time t_0; nothing is copied. Let $s_t(o)$ be the stored row of object o at time t, with $s_t(o) = \varnothing$ if o does not exist. Before the first editor change to o after t_0, the

journal records

$$J_B(o) = \begin{cases} s_{t_0}(o) & \text{if } o \text{ is edited or deleted} \\ \varnothing & \text{if } o \text{ is created} \end{cases} \qquad \text{only if } o \notin \mathrm{dom}(J_B), \tag{88}$$

i.e. the first pre-image wins: an object edited n times is still restored to its state at t_0. Rows are serialized with Django's serializer, which keeps the geometry in its stored CRS. The bar in the Layers panel reports $|\mathrm{dom}(J_B)|$ as "objects changed". Without an active backup, J is not written and edits are permanent. At most one backup exists at a time.

7.6 Restore — `restore_backup()`

Location: `mainMap/editing.py:269`

In one transaction (wrapped in `importer/batching.py::deferred_cascades()`), for every $o \in \mathrm{dom}(J_B)$:

$$s_{\text{after}}(o) = J_B(o) = \begin{cases} s_{t_0}(o) & \text{(raw save: update, or re-insert with the original id)} \\ \varnothing & \text{(delete the object created after } t_0\text{)} \end{cases} \tag{89}$$

Re-insertions and updates run before deletions so that reference targets exist again first. The raw save writes the stored values back exactly, derived fields included, without re-running `save()`, and still fires `post_save`, which bumps the layer's GeoJSON cache version. If `_check_constraints()` fails, the whole restore is rolled back (never partial); otherwise B and J_B are deleted and the affected layer keys are returned for the map to reload. Objects changed only outside the editor (importer, admin, QGIS, psql) are not in $\mathrm{dom}(J_B)$ and are left as they are.

Caveat: receivers that skip raw saves (`watersupply/signals.py`: `if raw: return`) would not fire on restore. Built-up models have none; adding such a layer to $\mathcal{E}$ requires triggering its cascades explicitly.

Side effects of an edit. Rows that another app changes because of an edit are journalled too, through custom signals that `mainMap/signals.py` subscribes to. Creating a building runs `urbanize_around()` (§2.9), which sends `landcover_rewriting` with the land-cover rows $\mathcal{A}$ it is about to clip or delete and `landcover_created` with the new urban-fabric row u; the receivers add $J_B(p) = s_{t_0}(p)$ for $p \in \mathcal{A}$ and $J_B(u) = \varnothing$ under the layer key `physicalEnv.LandCoverVector`. Restore therefore re-inserts the clipped and deleted polygons with their original geometry and deletes u, and the land-cover receiver recomputes the neighbourhoods' urban area inside the same deferred block.

7.7 Map-side editor — `Editing.js`

Location: `core/static/js/Editing.js`; hooks in `Layers.js` (`reloadLayer()`, `_popupEditButton()`), `MapUI.js::renderLayerList()`, `mainMap.js::refreshActivePanel()`, `Map_init.js`

No equations; behaviour relevant to reproducing results:

- `startEdit()` / `startCreate()` load the form into the side panel; a `MutationObserver` on `#panel-body` attaches Mapbox GL Draw when the form appears and detaches it when the panel content is replaced.
- While editing, the original object is hidden with the filter `id` $\neq$ pk, feature popups, admin-unit clicks and panel refreshes are suspended, and the camera pitch is set to 0.
- The `feature-saved` and `backup-restored` HTMX events reload the affected layers (`reloadLayer()`), so the map always shows the stored state.

8 Imported climate-effect indicators (importer app, Klimaateffectatlas)

Source: `importer/external_catalog.py` (`kea_*` entries, `_kea_neighborhood()`, `_kea_building()`), `importer/external_data.py` (`_prepare_features()`, `_keep_existing()`, `_enrich_rows()`); target models `builtup.NeighborhoodClimateRisk`, `urban_heat.NeighborhoodHeat`, `housing.NeighborhoodSocialPro` and new `builtup.Building` fields. The layer-by-layer mapping, and the 47 of 68 WFS layers that are not imported, are listed in `docs/KEA_WFS_MAPPING.md`. No equation of CrossTwin produces these values: they are stored inputs, transformed only on import.

8.1 Value transforms — `_prepare_features()`

For a source attribute x of feature f:

$$x' = \begin{cases} \varnothing & x \in \mathcal{N} \\ m(x) & x \in \mathrm{dom}(m) \\ k\,x & \text{for a scaled attribute} \\ x & \text{otherwise} \end{cases} \tag{90}$$

with $\mathcal{N} = \{-99995, \ldots, -99999\}$, the RIVM health monitor's no-value codes (frail health, loneliness, rental share, socio-economic status); $k = 100$ for the pile-rot risks, which KEA delivers as fractions (0–0.054 in Enschede), while settlement risk is already a percentage (0.9–17.7) and is stored as delivered; and m the drought-sensitivity legend, $m(\mathrm{W}) = 2$ (weinig gevoelig, Low), $m(\mathrm{G}) = 3$ (gevoelig, Medium), $m(\mathrm{Z}) = 4$ (zeer gevoelig, High), with a missing value left to the model default 0 (not groundwater-dependent). A building-block feature whose `BAG.Id` lists n panden is split into n features, each id left-padded to the 16 digits of a BAG identifier, so that every pand of the block receives the block's facade-shade score.

8.2 Join and write rules — `_keep_existing()`, `_enrich_rows()`

Let K be the set of stored keys of the join model (`Neighborhood.id` or `Building.id`) and $k(f)$ the feature's key (`buurtcode`, `identificatie`, ...). Only features with $k(f) \in K$ are written; the others are counted in the import message. Neighbourhood layers upsert one row per neighbourhood on `neighborhood_id`, each layer writing only its own columns. Building layers update the existing row only, $s'(b) = s(b) \oplus \{\text{mapped attributes of } f\}$, never insert and never write the geometry; they use `update()` and bump the map cache of every registry model on the building table (including the `Energy.BuildingEnergyLabel` proxy).

DAG edges: none computed. The values inform `PET`, `Vegetation_Coverage` (heat and canopy), and the context of `Income_Expenses`/`Affordability_Stress` (social profile) — stored inputs.

9 Summary table

App	Function	DAG edge(s) (abbrev.)	Status
administrative	`extrapolate()`	Population_Growth→TotalPop	implemented
administrative	`_city_growth_factor()`	Population_Growth→TotalPop	implemented

App	Function	DAG edge(s) (abbrev.)	Status
administrative	`annual_growth_rate()`	Population_Growth→Total_Pop	implemented
administrative	`City.save()` / `Province.save()`	(population cascade)	implemented
physicalEnv	`urbanize_around()` / `building_created`	New_Units→LandCover (editor buildings)	partial
watersupply	`calculate_total_demand()` / `TotalWaterDemand.save()`	Total_Pop→Total_Water_Demand	implemented
watersupply	`ConsumptionCapita.save()`	Total_Pop→Total_Consumption	implemented
watersupply	`calculate_total_extraction()`	Available_FW→Total_Extraction	implemented
watersupply	`calculate_total_production()`	Total_Extraction/Imported→Total_Water_Prod	implemented
watersupply	`calculate_supply_security()` / `SupplySecurity.save()`	Total_Water_Demand/Prod→Supply_Security→Service_T	implemented
watersupply	`calculate_energy_consumption()`	Total_Extraction/Prod→Energy_Consumption	partial
watersupply	`ExtractionWater.save()` (emissions)	Total_Extraction→CO2_Emission	implemented
watersupply	`OPEX.save()`	Extraction/Imported/WT_Cost/Network→OPEX	implemented
watersupply	`calculate_opex_recovery()`	OPEX/CollectionRatio/NRW→OPEX_Recovery	partial
watersupply	`MeteredResidential.save()`	Metered_Res_Water→OPEX_Recovery	implemented
watersupply	`calculate_collection_ratio()`	Metered_Res_Water/User_Acceptance/Water_Tariff→Col	partial
watersupply	`calculate_coverage()` / `CoverageWaterSupply.save()`	Network/CityArea→Coverage_WS_Area	implemented
watersupply	`calculate_nrw()` / `NonRevenueWater.save()`	Real/Apparent_Losses→NRW, ILI	partial
watersupply	`calculate_water_quality()`	Samples_WQ→User_Acceptance_WS	gap
watersupply	`calculate_available_freshwater()`	Infiltration/Meteorology→Available_FW	gap
physicalEnv	`classify_soil_group()` / `SoilType.save()`	Soil_Type→Infiltration	implemented
physicalEnv	`soil_landcover_composition()`	LandCover/Soil_Type (+ groundwater)→Infiltration	implemented
physicalEnv	`scs_runoff_mm()` / `summarize_infiltration()`	LandCover/Soil_Type→Infiltration	implemented
watersupply	`calculate_infiltration()` (dashboard)	LandCover/Soil_Type→Infiltration	implemented
housing	`calculate_supply_demand()`	Urbanization→Housing_Demand	gap
housing	`calculate_new_units()`	New_Units→Housing_Supply/LandCover	partial
housing	`calculate_mortgage_indicators()`	Central_Bank/Credit_Supply→Mortgage	partial
housing	`calculate_house_price_index()`	Housing_Demand/Property→HPI	gap
housing	`HousingAffordability.save()`	Rent/Mortgage/Water_Tariff→Income_Expenses→Afford	gap
housing	`calculate_zoning()`	Zoning_Status→Property/New_Units	gap
housing	`calculate_property_indicators()`	Housing_Supply/Accessibility→Property	gap
urban_heat	`calculate_green_area()` / `calculate_vegetation_coverage()`	LandCover→Green_Area→Vegetation_Coverage	partial

App	Function	DAG edge(s) (abbrev.)	Status
urban_heat	`calculate_urban_morphology()`	Buildings/Streets→Canyon Aspect	partial
urban_heat	`get_thermal_indices()`	DSM→SVF→Tmrt/PET→UTCI	gap
urban_heat	`calculate_nbs_coverage()`	UTCI/PET→NBS	gap
mainMap	`feature_form()` / `feature_delete()`	(tooling: edits State objects via `save()`)	implemented
mainMap	`journal()` / `restore_backup()`	(tooling: what-if restore point)	implemented
importer	`_prepare_features()` / `_enrich_rows()` (KEA)	(stored inputs; no edge)	implemented

Notes on use

This document is generated from a manual audit of the codebase at the commit referenced in the accompanying `TODO.md`. Line numbers will drift as the code evolves; re-verify against `grep -n "def save"` in the relevant module before citing a location in a manuscript. The status tags mirror the coverage figures reported in the project's DPSIR gap analysis: of 103 edges in `core/DAG.dot` (one, `Total_Water_Prod`→`OPEX`, is listed twice), 31 are backed by real derivation logic as of this audit; the three added since the original audit are `LandCover`→`Infiltration`, the new `Soil_Type`→`Infiltration`, and `New_Units`→`LandCover` for buildings drawn in the map editor.

Beyond the CBS 85173NED forecasts used for the `Population_Growth`→`Total_Population` projection, the external data catalog also offers a finer-grained, Google Earth Engine-hosted population source, the High Resolution Settlement Layer (HRSL) [2], for direct raster ingestion where a sub-neighbourhood estimate is preferable to the CBS city-level forecast.

References

[1] Centraal Bureau voor de Statistiek (CBS). *Regionale prognose 2023–2050* (Table 85173NED). `https://opendata.cbs.nl/`.

[2] Facebook Connectivity Lab and Center for International Earth Science Information Network (CIESIN), Columbia University (2016). *High Resolution Settlement Layer (HRSL).* Source imagery for HRSL Copyright 2016 DigitalGlobe. `https://gee-community-catalog.org/projects/hrsl/`.

[3] Lambert, A., Brown, T. G., Takizawa, M., & Weimer, D. (1999). A review of performance indicators for real losses from water supply systems. *Journal of Water Supply: Research and Technology-AQUA*, 48(6), 227–237. (International Water Association Water Loss Task Force ILI methodology.)

[4] Demographia. *International Housing Affordability Survey* — Median Multiple methodology (median house price / median household income). `http://www.demographia.com/`.

[5] Lindberg, F., Holmer, B., & Thorsson, S. (2008). SOLWEIG 1.0 – Modelling spatial variations of 3D radiant fluxes and mean radiant temperature in complex urban settings. *International Journal of Biometeorology*, 52(7), 697–713.

[6] Oke, T. R. (1987). *Boundary Layer Climates* (2nd ed.). Routledge. (Urban canyon height-to-width aspect ratio.)

[7] Matzarakis, A., & Mayer, H. (1996). Another kind of environmental stress: thermal stress. *WHO Newsletter*, 18, 7–10. (Physiological Equivalent Temperature classification.)

[8] Bröde, P., Fiala, D., Błażejczyk, K., et al. (2012). Deriving the operational procedure for the Universal Thermal Climate Index (UTCI). *International Journal of Biometeorology*, 56(3), 481–494.

[9] USDA Natural Resources Conservation Service (1986). *Urban Hydrology for Small Watersheds*. Technical Release 55 (TR-55), 2nd ed. (Curve number tables for antecedent moisture condition II.)

[10] USDA Natural Resources Conservation Service (2009). *National Engineering Handbook, Part 630 Hydrology, Chapter 7: Hydrologic Soil Groups*. (Group definitions, final infiltration rates, dual groups for a water table within 60 cm.)

[11] USDA Natural Resources Conservation Service (2004). *National Engineering Handbook, Part 630 Hydrology, Chapter 10: Estimation of Direct Runoff from Storm Rainfall*. (Runoff equation, $I_a = 0.2S$.)

[12] Wageningen Environmental Research. *Bodemkundig Informatie Systeem (BIS Nederland)*: Bodemkaart 1:50 000 (2025) and groundwater-level grids (GHG, GLG, GVG). `https://bodemdata.nl/`.

[13] European Environment Agency. *CORINE Land Cover nomenclature*. `https://land.copernicus.eu/`.

[14] European Commission. *INSPIRE Data Specification on Land Use (D2.8.III.4)*, Hierarchical INSPIRE Land Use Classification System (HILUCS). `http://inspire.ec.europa.eu/codelist/HILUCSValue`.